Data Structures and Algorithms I.



University of Szeged

Robert Nagy

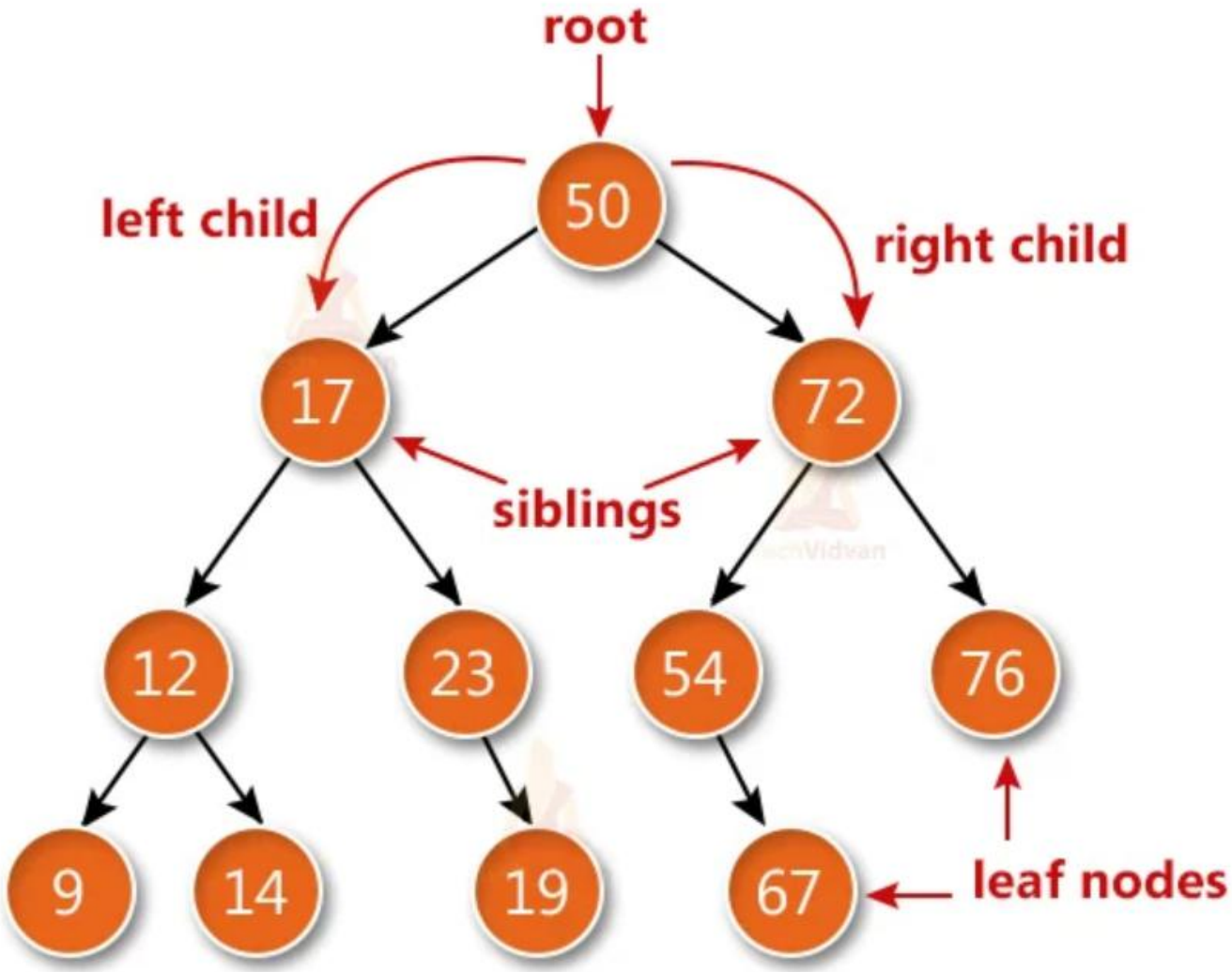
2025/26. Autumn Semester



Tree-like structures

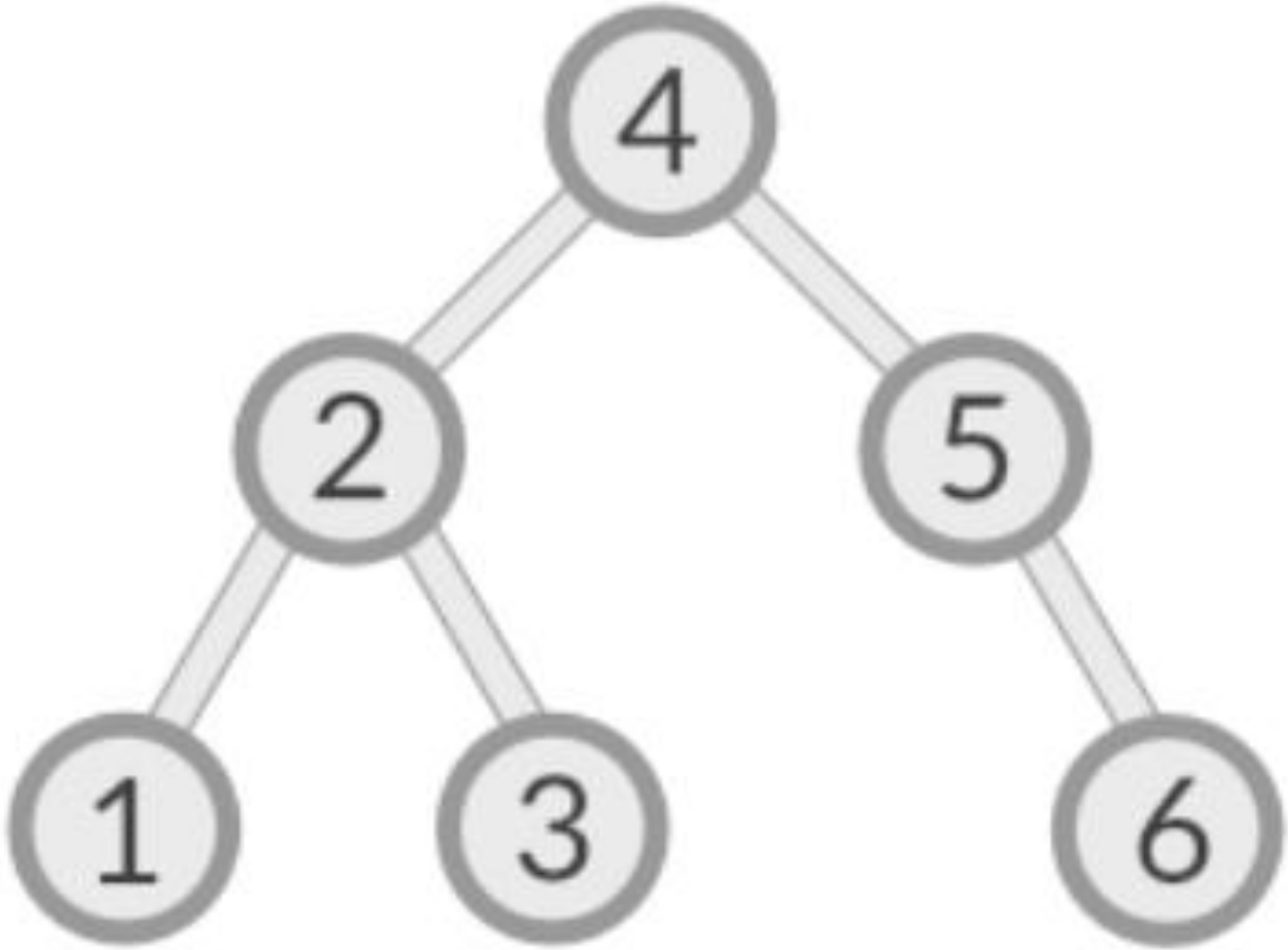
- We have already seen
  - Recursion tree
  - Heap

Binary Search Tree

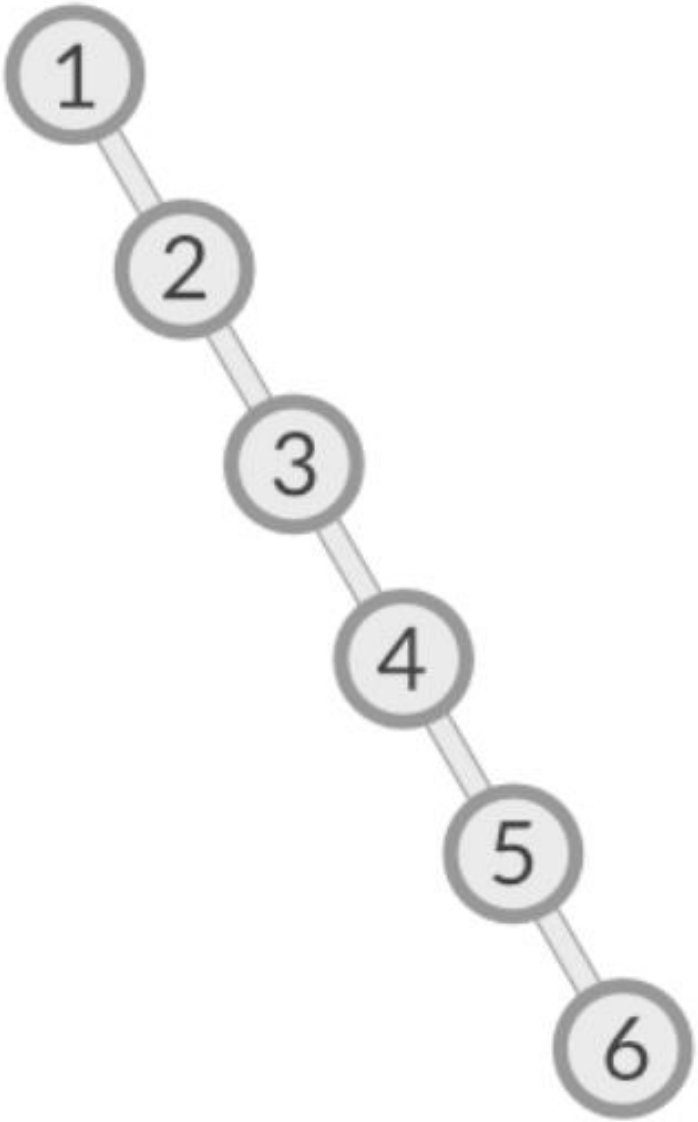


Different types of binary trees

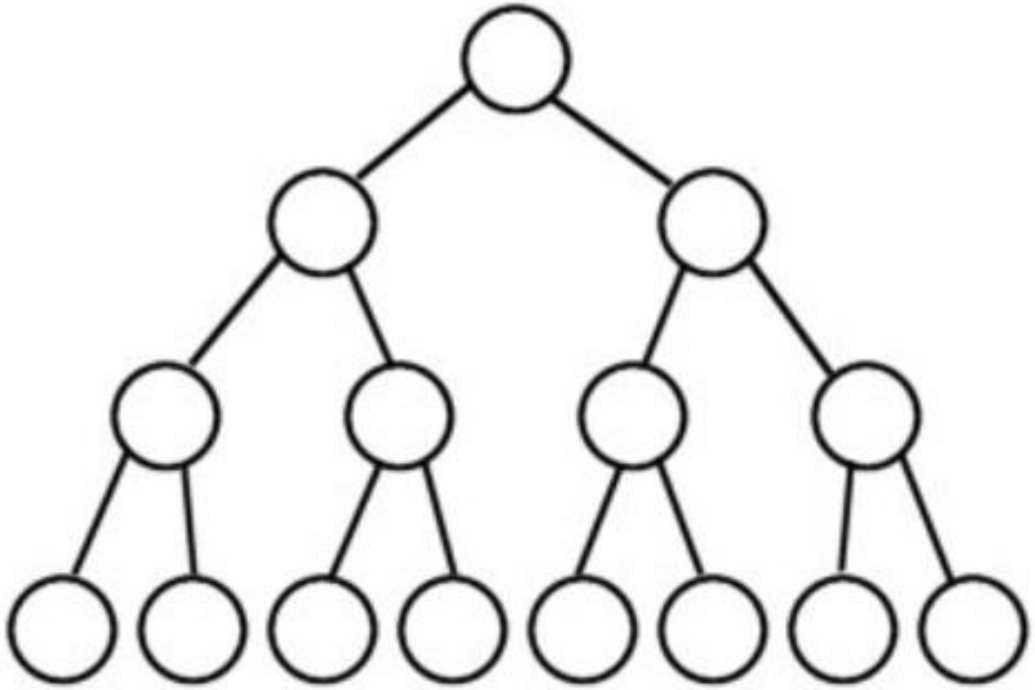
Balanced



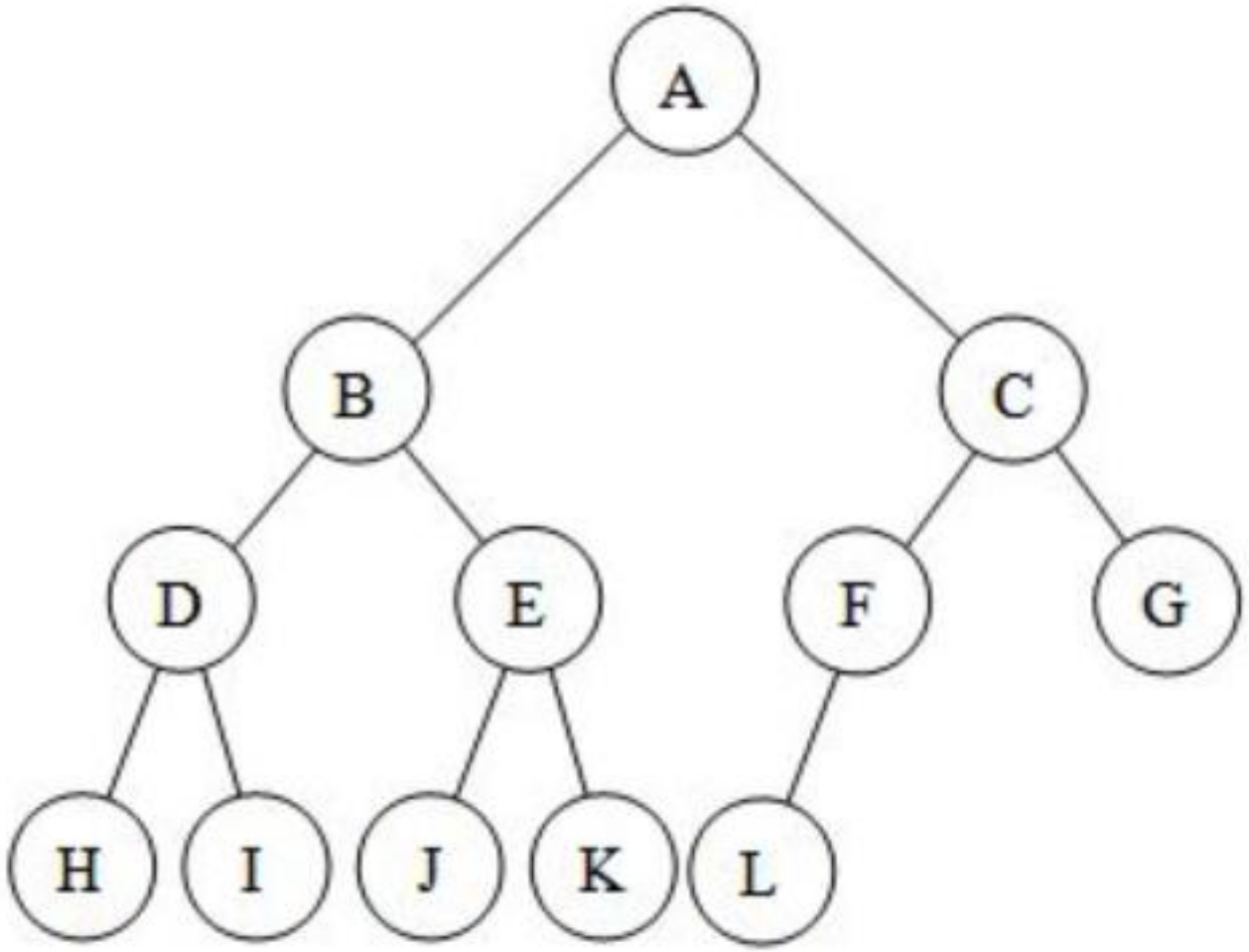
Skewed



Full binary tree



Complete binary tree



Exercise

- The following data stream arrives: 32, 15, 77, 65, 43, 6, 9
  - Construct a binary search tree, starting with the first element from left to right.

Ways to traverse a binary tree

```
def printInorder(node):
    if node is None:
        return

    # First recur on left subtree
    printInorder(node.left)

    # Now deal with the node
    print(node.data, end=' ')

    # Then recur on right subtree
    printInorder(node.right)
```

In-order

Post-order

```
def print_postorder(node):
    if node is None:
        return

    # First recur on left subtree
    print_postorder(node.left)

    # Then recur on right subtree
    print_postorder(node.right)

    # Now deal with the node
    print(node.data, end=' ')
```

Pre-order

```
def printPreorder(node):
    if node is None:
        return

    # Deal with the node
    print(node.data, end=' ')

    # Recur on left subtree
    printPreorder(node.left)

    # Recur on right subtree
    printPreorder(node.right)
```

Set

Abstract data type

Unique elements

Ordered

Unordered

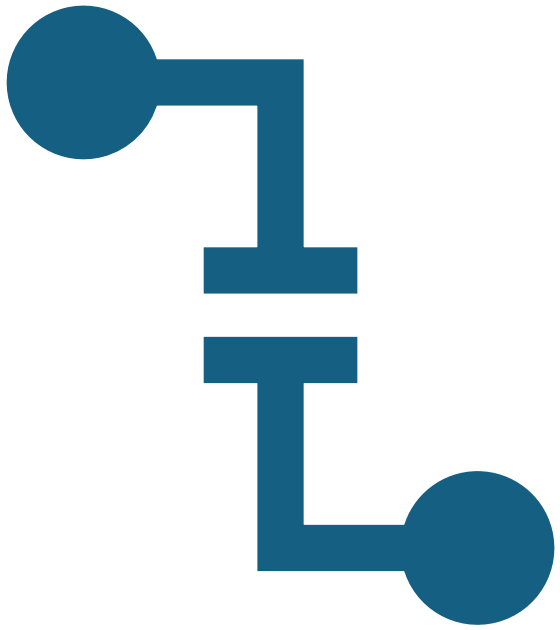
Map (Dictionary)

Key – value pairs

Map (Dictionary)

Ordered

Unordered



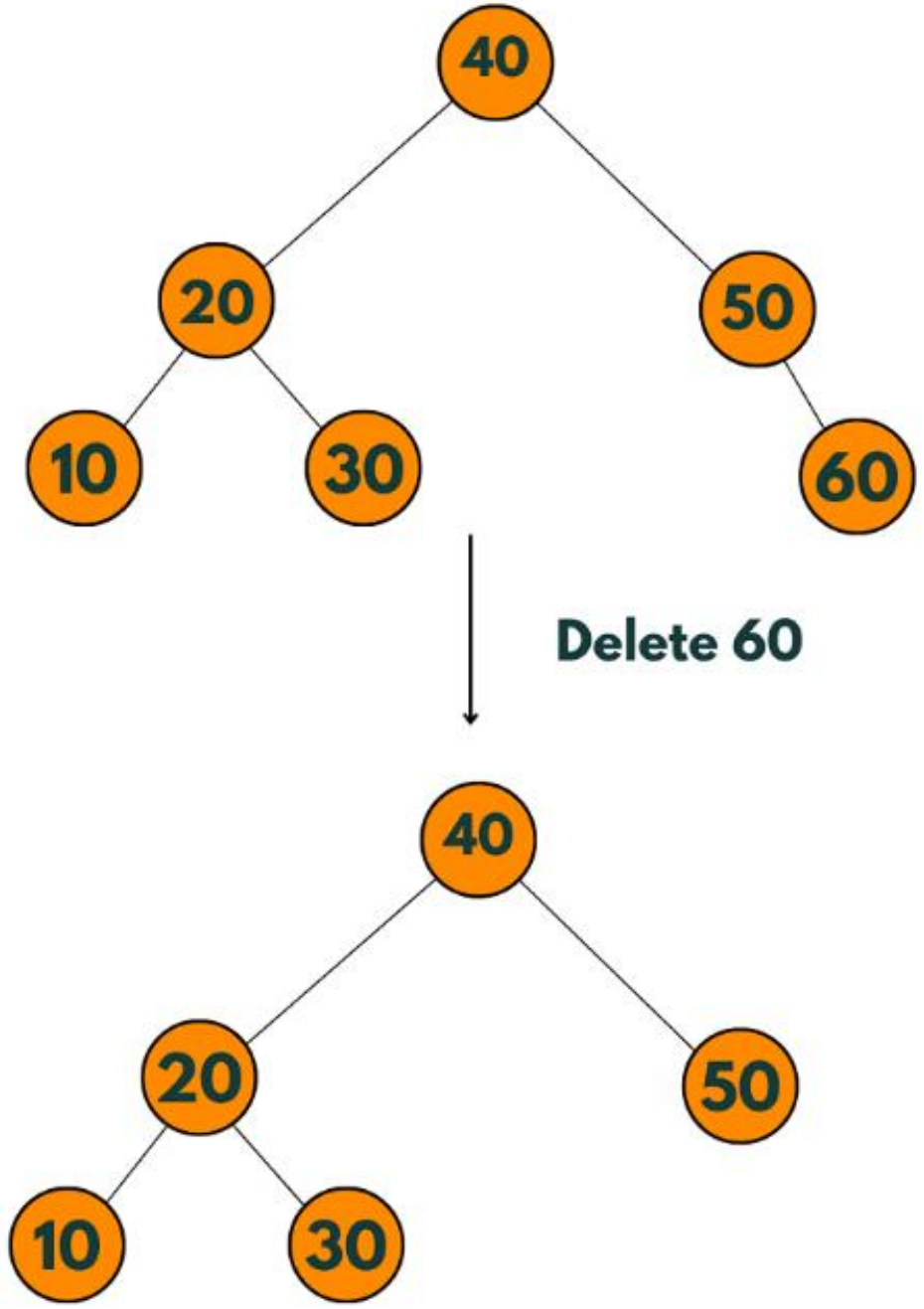
Deleting a node from a BST

2 distinct cases

Leaf nodes

Internal nodes

Leaf nodes



Internal nodes – 2 strategies

Replace with the previous

Replace with the next

Time complexity analysis

- Each operation depends on the height of the tree
  - Worst case – skewed → *O(n)*
  - Best case – balanced → *O(lg(n))*